AI in Fluids: Week 1

From conservation laws to a cavity solver

Numerical foundations for students new to CFD

The learning question. How can a computer predict the recirculating flow in a box when only the wall motion, fluid properties, and geometry are given?

After Week 1, you should be able to

1. Write continuity and both momentum components, and derive their nondimensional form.
2. Define ψ and ω, explain their physical meaning, and prove $\nabla^2\psi = -\omega$.
3. Explain why the vorticity equation needs velocities and how the coupled solve obtains them.
4. Convert a derivative and the Poisson equation into finite differences, keeping array axes consistent.
5. Distinguish inner Poisson iteration, outer time evolution, stability, convergence, and validation.
6. Compare centerline profiles quantitatively and describe the limits of that evidence.

A neural network can fit a field without ensuring that it conserves mass or satisfies a wall condition. This week establishes the physical and numerical language needed to judge both CFD solutions and later machine-learning predictions. We start with conservation of mass and momentum, derive a pressure-free formulation for two-dimensional incompressible flow, and connect each equation to the existing cavity notebook.

Three companion notebooks

1. `01_python_for_cfd_ai_fluids.ipynb`: variables, functions, arrays, derivatives, and residuals.
2. `02_tensorflow_for_ai_fluids.ipynb`: tensors, shapes, losses, automatic differentiation, and a small neural network.
3. `03_cavity_ghia.ipynb`: a finite-difference streamfunction–vorticity solver and centerline comparison at $Re = 100$.

All three are in `notebooks/week01/`. This lecture explains the introductory $Re = 100$ solver; retained pressure-validation demonstrations elsewhere in the notebook have their own conditions and evidence.

Reading route

Sections	Topic
1–3	Fluid fields, conservation laws, and nondimensional equations
4–6	Streamfunction, vorticity, and the pressure-free formulation
7–7.3	Finite differences: Taylor series, errors, stencils, and a coupled system
8–10	Wall conditions, time integration, convergence, and stability
11	Benchmark comparison and assessment
12–12.2	Optional pressure derivation, wall data, and recovery
B1–B4	Code applications of the preceding derivations
C–D	Algorithm diagrams and topic coverage

Conventions. A right-handed (x, y, z) system is used, with x rightward and y upward. Positive $\omega = \omega_z$ means counterclockwise local rotation. Dimensional quantities have a subscript d when scales are introduced. After nondimensionalization, unadorned variables are dimensionless.

Scope. A constant-density, constant-viscosity Newtonian fluid; two-dimensional incompressible motion; no nonconservative body force. The cavity is a square with a uniformly translating top wall. This is a teaching model, not a model for rarefied or compressible flow.

1 Fluids, fields, and acceleration

A fluid continuously deforms under an applied shear stress. A solid can sustain a static shear deformation; a fluid keeps deforming under sustained shear. Air, water, oil, and gas mixtures are examples. Both liquids and gases are fluids, although their density and compressibility differ. In the continuum description, density, velocity, pressure, and temperature are fields rather than properties of individually tracked molecules. The approximation requires molecular length scales to be small compared with the relevant flow scales.

Symbol	Meaning	SI unit
ρ	Density	$\mathrm{kg\,m^{-3}}$
$\boldsymbol{u}_d = (u_d, v_d)$	Velocity	$\mathrm{m\,s^{-1}}$
p_d	Pressure	Pa
μ	Dynamic viscosity	Pa s
$\nu = \mu/\rho$	Kinematic viscosity	$\mathrm{m^2\,s^{-1}}$
T	Temperature (thermal state)	K
ψ_d	Two-dimensional streamfunction	$\mathrm{m^2\,s^{-1}}$
ω_d	Out-of-plane vorticity	$\mathrm{s^{-1}}$

Here ρ is mass per unit volume, p_d is isotropic compressive normal stress, μ measures resistance to shear deformation, and ν is momentum diffusivity. Temperature is not solved in this constant-property cavity model.

In an Eulerian description, $u_d = u_d(x_d, y_d, t_d)$: we ask what passes a fixed position. A Lagrangian description instead follows a moving particle. The cavity notebook uses Eulerian arrays, with `u[j,i]` representing the horizontal velocity at (x_i, y_j).

Why acceleration is not just a time derivative. Follow a fluid particle with $\mathrm{d}x_d/\mathrm{d}t_d = u_d$ and $\mathrm{d}y_d/\mathrm{d}t_d = v_d$. Applying the chain rule to its horizontal velocity gives

$$\frac{Du_d}{Dt_d} = \frac{\partial u_d}{\partial t_d} + u_d\frac{\partial u_d}{\partial x_d} + v_d\frac{\partial u_d}{\partial y_d}. \tag{1}$$

The first term describes change at a fixed point; the last two describe change as the particle moves through a nonuniform field. A steady flow can therefore have nonzero acceleration. The vector statement is

$$\frac{D\boldsymbol{u}_d}{Dt_d} = \frac{\partial \boldsymbol{u}_d}{\partial t_d} + (\boldsymbol{u}_d \cdot \nabla_d)\boldsymbol{u}_d. \tag{2}$$

From physics to arrays

- A continuous field becomes values at grid nodes, not a photograph of exact values everywhere.
- A spatial derivative becomes a difference between neighboring entries.
- Boundary conditions become constraints on boundary entries.
- A residual measures an equation defect or an iteration change; neither is automatically the error relative to the exact solution.

Check your understanding. Can a particle accelerate when $\partial \boldsymbol{u}_d/\partial t_d = 0$? Explain using a particle turning around a cavity vortex.

2 Review: continuity and momentum

The incompressible Navier–Stokes system. In this course this means solving conservation of mass (continuity) together with both momentum components. The three unknowns are horizontal velocity, vertical velocity, and pressure. Momentum alone does not enforce mass conservation; continuity alone does not determine motion. Initial data and wall conditions complete the problem. We first write the dimensional system, then derive the dimensionless equations actually used in the code.

Mass conservation. The general continuity equation is

$$\frac{\partial \rho}{\partial t_d} + \nabla_d \cdot (\rho \boldsymbol{u}_d) = 0. \tag{3}$$

For the constant-density fluid used here, it becomes

$$\boxed{\frac{\partial u_d}{\partial x_d} + \frac{\partial v_d}{\partial y_d} = 0.} \tag{4}$$

A small material fluid element preserves its volume: local inflow and outflow balance. This does not require either velocity component to be spatially constant.

Momentum conservation. For an incompressible Newtonian fluid with constant properties and no body force, the vector statement is $\partial_t \boldsymbol{u}_d + (\boldsymbol{u}_d \cdot \nabla_d)\boldsymbol{u}_d = -\rho^{-1}\nabla_d p_d + \nu \nabla_d^2 \boldsymbol{u}_d$, where the time derivative is with respect to t_d. Written componentwise:

$$\underbrace{\frac{\partial u_d}{\partial t_d}}_{\text{local change}} + \underbrace{u_d \frac{\partial u_d}{\partial x_d} + v_d \frac{\partial u_d}{\partial y_d}}_{\text{convection}} = \underbrace{-\frac{1}{\rho}\frac{\partial p_d}{\partial x_d}}_{\text{pressure force}} + \underbrace{\nu \left(\frac{\partial^2 u_d}{\partial x_d^2} + \frac{\partial^2 u_d}{\partial y_d^2} \right)}_{\text{viscous diffusion}}, \tag{5}$$

$$\frac{\partial v_d}{\partial t_d} + u_d \frac{\partial v_d}{\partial x_d} + v_d \frac{\partial v_d}{\partial y_d} = -\frac{1}{\rho}\frac{\partial p_d}{\partial y_d} + \nu \left(\frac{\partial^2 v_d}{\partial x_d^2} + \frac{\partial^2 v_d}{\partial y_d^2} \right). \tag{6}$$

Every term has units of acceleration. Pressure gradients push fluid from high toward low pressure; viscosity diffuses momentum between neighboring regions. Convection is nonlinear because the unknown velocity multiplies derivatives of the unknown velocity.

Three unknown fields. Equations (4)–(6) couple u_d, v_d, and p_d. Pressure-based CFD algorithms can solve this system directly while enforcing incompressibility. In this lecture we transform it into a system for ψ and ω that is especially convenient for a two-dimensional simply connected cavity. The transformed system still represents momentum conservation.

Initial and boundary data. A time-dependent problem also needs an initial velocity field consistent with incompressibility and the prescribed conditions. The teaching solver starts from zero interior fields and applies the moving lid impulsively. The initial boundary jump creates a transient before the flow approaches a steady state.

Check your understanding. Write the vertical convective acceleration. Why would using only the horizontal momentum equation leave the cavity problem incomplete?

3 Deriving the nondimensional equations

Let L be the cavity side length and $U > 0$ the lid-speed scale. Choose the convective time L/U and dynamic pressure ρU^2. Define

$$x = \frac{x_d}{L}, \quad y = \frac{y_d}{L}, \quad t = \frac{t_d U}{L}, \quad u = \frac{u_d}{U}, \quad v = \frac{v_d}{U}, \quad p = \frac{p_d - p_{\text{ref}}}{\rho U^2}. \tag{7}$$

The newly defined variables x, y, t, u, v, p are dimensionless. The chain rule gives, for example,

$$\frac{\partial u_d}{\partial t_d} = \frac{U^2}{L} u_t, \qquad u_d \frac{\partial u_d}{\partial x_d} = \frac{U^2}{L} u u_x,$$

$$\frac{1}{\rho}(p_d)_{x_d} = \frac{U^2}{L} p_x, \qquad \nu \nabla_d^2 u_d = \frac{\nu U}{L^2} \nabla^2 u.$$

Divide horizontal momentum by U^2/L. The coefficient of diffusion is $\nu/(UL) = 1/Re$, where

$$\boxed{Re = \frac{UL}{\nu} = \frac{\rho UL}{\mu}.} \tag{8}$$

The dimensionless system, with subscripts denoting derivatives, is

$$u_x + v_y = 0, \tag{9}$$

$$u_t + u u_x + v u_y = -p_x + \frac{1}{Re}(u_{xx} + u_{yy}), \tag{10}$$

$$v_t + u v_x + v v_y = -p_y + \frac{1}{Re}(v_{xx} + v_{yy}). \tag{11}$$

Interpretation. Reynolds number compares inertial and viscous effects using the chosen scales. Its value alone does not give a universal transition threshold: geometry, disturbances, and boundary conditions also matter. At small Re, viscous diffusion dominates; increasing inertia can produce recirculation and thinner layers. At sufficiently large Re, instability, transition, and turbulence may occur, depending on the flow. These are tendencies, not universal Reynolds-number thresholds.

Notebook translation. The square becomes $0 \le x, y \le 1$, and the lid speed is $u = 1$. At $Re = 100$, the dimensionless diffusion coefficient is 0.01. This does *not* imply a physical kinematic viscosity of $0.01\ \text{m}^2/\text{s}$ unless the dimensional scales make that true. For example, $L = 0.1$ m and $U = 0.01$ m/s give $\nu = 10^{-5}\ \text{m}^2/\text{s}$ at the same Reynolds number.

Two more scales. Set $\psi = \psi_d/(UL)$ and $\omega = \omega_d L/U$. These choices preserve $u = \psi_y$, $v = -\psi_x$, and $\omega = v_x - u_y$. All equations on the following pages use dimensionless variables unless stated otherwise.

Check your understanding. Which time in seconds corresponds to dimensionless $t = 10$ for the example above? Answer: 100 s.

4 Streamfunction and vorticity: definitions

For a sufficiently smooth, two-dimensional incompressible velocity field in this simply connected domain, introduce a scalar streamfunction ψ through

$$\boxed{u = \psi_y, \qquad v = -\psi_x.} \tag{12}$$

Substitution into continuity gives

$$u_x + v_y = \psi_{yx} - \psi_{xy} = 0, \tag{13}$$

because the mixed derivatives commute. Thus the streamfunction representation builds mass conservation into the continuous equations.

Why its contours are streamlines. At a fixed instant,

$$\boldsymbol{u} \cdot \nabla\psi = u\psi_x + v\psi_y = \psi_y\psi_x - \psi_x\psi_y = 0. \tag{14}$$

Velocity is tangent to a contour ψ = constant. For unsteady flow these instantaneous streamlines need not be particle paths. At fixed x, the volume flow rate per unit depth between y_1 and y_2 is $\int_{y_1}^{y_2} u\,\mathrm{d}y = \psi(x, y_2) - \psi(x, y_1)$.

Vorticity. The curl of velocity has only a z component in this two-dimensional flow:

$$\boxed{\omega = v_x - u_y.} \tag{15}$$

Vorticity measures local rotation; the local rigid-body angular velocity is $\omega/2$. It is not speed, and it is not the curvature of a streamline. A shear flow can have nonzero vorticity even with straight streamlines.

Example: Couette shear. For $u = y$, $v = 0$, choose $\psi = y^2/2$. Then $u = \psi_y$, $v = -\psi_x$, and $\omega = -1$. Horizontal layers move at different speeds, producing clockwise local rotation. This sign convention is also consistent with the negative wall vorticity generated by a rightward-moving cavity lid.

A useful numerical distinction. Central differences for $u = D_y\psi$ and $v = -D_x\psi$ give a zero discrete divergence wherever the same commuting stencils are used. Overwriting wall velocities or switching stencils near boundaries requires separate checks; the continuum identity is not a guarantee that every boundary discretization is consistent.

Check your understanding. For solid-body rotation $u = -\Omega y$, $v = \Omega x$, find ω and a compatible ψ. Answer: 2Ω and $-\Omega(x^2 + y^2)/2$ plus a constant.

5 Proof of the streamfunction Poisson equation

Goal: prove that the streamfunction satisfies $\nabla^2\psi = -\omega$. Begin with the definitions; no new physical assumption is needed beyond those already stated:

$$\omega = \frac{\partial v}{\partial x} - \frac{\partial u}{\partial y}, \tag{16}$$

$$v = -\psi_x \quad \Rightarrow \quad v_x = -\psi_{xx}, \tag{17}$$

$$u = \psi_y \quad \Rightarrow \quad u_y = \psi_{yy}. \tag{18}$$

Therefore

$$\omega = -\psi_{xx} - \psi_{yy} = -\nabla^2\psi, \tag{19}$$

and hence

$$\boxed{\nabla^2\psi = -\omega.} \tag{20}$$

The minus sign follows from the paired conventions $u = \psi_y$, $v = -\psi_x$ and $\omega = v_x - u_y$. Changing only one convention would change the problem incorrectly.

What this equation does. If ω is known throughout the interior and appropriate wall values of ψ are prescribed, Eq. (20) is an elliptic Poisson problem for ψ. Differentiating its solution then gives velocity. This is a spatial solve at each stage of the algorithm; the equation itself contains no time derivative.

A manufactured check. Let

$$\psi = \sin(\pi x)\sin(\pi y). \tag{21}$$

Then

$$u = \pi \sin(\pi x)\cos(\pi y), \qquad v = -\pi \cos(\pi x)\sin(\pi y), \tag{22}$$

$$\omega = 2\pi^2 \sin(\pi x)\sin(\pi y), \qquad \nabla^2\psi = -2\pi^2 \sin(\pi x)\sin(\pi y). \tag{23}$$

This verifies the sign and provides smooth fields for checking derivative code. It is a manufactured identity check, *not* a no-slip cavity solution: although $\psi = 0$ on the boundary, tangential velocities generally do not vanish there.

Why $\psi = 0$ on the cavity walls? Impermeability makes ψ constant along each wall. The connected cavity boundary permits one common constant; choose zero because adding a constant leaves velocity unchanged. Tangential no-slip is additional information, enforced through the wall-vorticity relation on page 13. Specifying $\psi = 0$ alone does not impose a stationary wall.

Check your understanding. Explain why solving the Poisson equation once cannot determine the flow if its right-hand side ω is still unknown.

6 Optional derivation: vorticity transport

You may skip the product-rule steps on a first reading, but read the boxed transport equation and its physical interpretation: both are needed for the time update in Section 9. Use the dimensionless momentum equations from page 4. Take ∂_x of vertical momentum and subtract ∂_y of horizontal momentum. Smoothness allows mixed derivatives to commute.

Step 1: time and pressure.

$$\partial_x v_t - \partial_y u_t = \omega_t, \qquad -p_{yx} + p_{xy} = 0.$$

Pressure disappears because the curl of a gradient is zero; pressure itself need not be uniform or zero.

Step 2: expand convection. Applying the product rule gives

$$\begin{aligned}\partial_x(uv_x + vv_y) - \partial_y(uu_x + vu_y) &= u(v_{xx} - u_{xy}) + v(v_{xy} - u_{yy}) \\ &\quad + u_x v_x + v_x v_y - u_y u_x - v_y u_y \\ &= u\omega_x + v\omega_y + (u_x + v_y)(v_x - u_y) \\ &= u\omega_x + v\omega_y.\end{aligned}$$

The last equality uses continuity, $u_x + v_y = 0$. More explicitly, the last four product-rule terms factor as

$$u_x v_x + v_x v_y - u_y u_x - v_y u_y = (u_x + v_y)(v_x - u_y) = 0 \cdot \omega = 0.$$

Their sum vanishes; none of these individual products is required to be zero. This is the cancellation that should not be skipped when deriving the equation for the first time.

Step 3: viscous diffusion. Since Re is constant,

$$\frac{1}{Re}\left[\partial_x \nabla^2 v - \partial_y \nabla^2 u\right] = \frac{1}{Re}\nabla^2 \omega.$$

Combining the three steps yields

$$\boxed{\omega_t + u\omega_x + v\omega_y = \frac{1}{Re}(\omega_{xx} + \omega_{yy}).} \tag{24}$$

In dimensional variables the diffusion coefficient is ν, not $1/Re$.

Physical reading. Local vorticity changes through advection by the velocity field and diffusion by viscosity. Moving and stationary no-slip walls supply boundary vorticity. There is no vortex-stretching term for the strictly two-dimensional velocity field assumed here. Compressible, variable-property, three-dimensional, or forced flows can require additional terms. In detail, the three-dimensional stretching term is $(\boldsymbol{\omega} \cdot \nabla)\boldsymbol{u}$. Here $\boldsymbol{\omega} = (0, 0, \omega)$ and $\partial_z \boldsymbol{u} = 0$, so this term equals $\omega \partial_z \boldsymbol{u} = 0$. This geometric argument is distinct from the continuity cancellation above. The viscous curl $Re^{-1}\nabla^2\omega$ does *not* vanish. Section 12 shows why the *divergence* of viscous diffusion vanishes in the pressure derivation; these are different operations.

Why this formulation helps. In the original equations, pressure adjusts the velocity to satisfy incompressibility and boundary conditions. Taking the curl removes its gradient from the transport equation above; defining a streamfunction then supplies the velocities while satisfying continuity. These are two distinct operations. The unknowns are now ψ and ω, coupled through Eq. (20); the pressure is not assumed zero. When pressure contours or forces are needed, recover pressure afterward as described in Section 12.

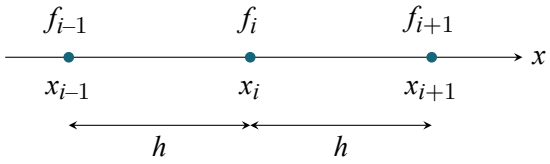
This is convenient for the simply connected two-dimensional cavity, but it still requires a Poisson solve and careful wall-vorticity data. It does not automatically improve speed or accuracy. A single scalar streamfunction is not a general three-dimensional velocity representation; pressure–velocity methods are useful alternatives for general geometries.

Check your understanding. Which operation removes pressure, and which representation satisfies continuity? The curl of momentum removes pressure; the streamfunction satisfies continuity.

7 Finite differences: from a slope to a formula

A computer stores finitely many samples of a continuous field. Finite differences turn derivatives into algebraic combinations of these samples. We derive the formulas, measure their errors, and assemble a Poisson equation.

Use a uniform grid $x_i = x_0 + ih$, with $h = x_{i+1} - x_i$, $f_i = f(x_i)$, and $f_i' = f'(x_i)$. A prime means differentiation, not an iteration. A derivative is a limiting slope, $f'(x_i) = \lim_{h\to 0}[f(x_i+h) - f(x_i)]/h$. On a grid, h is finite. We approximate the derivative using available samples and ask how the neglected terms depend on h. A difference is the numerator; a derivative approximation also divides by the correct distance.



What is a Taylor expansion? It approximates a smooth function near x_i by a polynomial. The term hf_i' predicts change using the slope; $h^2 f_i''/2$ corrects for curvature. The remainder $O(h^4)$ is bounded by a constant times h^4 for small h and bounded higher derivatives. Expanding on both sides gives

$$f_{i+1} = f_i + hf_i' + \frac{h^2}{2}f_i'' + \frac{h^3}{6}f_i''' + O(h^4),$$
$$f_{i-1} = f_i - hf_i' + \frac{h^2}{2}f_i'' - \frac{h^3}{6}f_i''' + O(h^4).$$

Subtract f_i from the first line and divide by h:

$$f_{i+1} - f_i = hf_i' + \tfrac{1}{2}h^2 f_i'' + \cdots \quad \Rightarrow \quad \frac{f_{i+1} - f_i}{h} = f_i' + \tfrac{1}{2}hf_i'' + \cdots .$$

The second expansion gives the backward formula; subtracting the expansions gives the centered formula. Error below means approximation minus exact derivative:

Method	Approximation to f_i'	Leading error
Forward	$(f_{i+1} - f_i)/h$	$+(h/2)f_i''$
Backward	$(f_i - f_{i-1})/h$	$-(h/2)f_i''$
Centered	$(f_{i+1} - f_{i-1})/(2h)$	$+(h^2/6)f_i'''$

Why $2h$? The centered numerator spans from $x_i - h$ to $x_i + h$, a distance $2h$. Dividing it by h would double the derivative.

Why use the centered formula here? For smooth interior fields with values on both sides, symmetry cancels the $O(h)$ term. The resulting $O(h^2)$ error is the reason for choosing it in this introductory diffusion and Poisson discretization. It does not guarantee stability for advection-dominated flows or shocks; a complete time-dependent scheme must also satisfy stability and boundary requirements.

Boundary distinction. At $i = 0$ there is no stored f_{-1}. A centered stencil therefore needs ghost values or a different boundary formula. A second-order one-sided derivative is $(-3f_0 + 4f_1 - f_2)/(2h)$. For cavity walls, Section 8 derives the boundary treatment.

Check your understanding. Which approximation needs the center value f_i, and which needs both neighbors? Do not confuse the first derivative with the three-point second derivative in Section 7.1.

7.1 Worked examples: accuracy and curvature

Take $f(x) = x^3$ at $x = 1$. The exact slope is $f'(1) = 3$. For $h = 0.1$, the samples are $f(0.9) = 0.729$, $f(1) = 1$, and $f(1.1) = 1.331$:

$$D_+ f = \frac{1.331 - 1}{0.1} = 3.31, \quad D_- f = \frac{1 - 0.729}{0.1} = 2.71, \quad D_0 f = \frac{1.331 - 0.729}{0.2} = 3.01.$$

h	Forward	Backward	Centered	$\lvert D_0 f - 3\rvert$
0.10	3.3100	2.7100	3.0100	0.0100
0.05	3.1525	2.8525	3.0025	0.0025

Halving h reduces the centered error by a factor of four here. This is what second-order truncation error means: approximately Ch^2 for a smooth fixed function as h decreases. It does not mean two decimal places or two iterations. At extremely small spacing, roundoff and subtractive cancellation can become important.

Derivative order versus accuracy order. A first derivative is a slope; a second derivative is curvature. Either can have second-order accuracy: this refers to how its error decreases with grid spacing, not which derivative we calculate.

Second derivative: add instead of subtract. Adding the Taylor expansions eliminates the odd powers:

$$f_{i+1} + f_{i-1} = 2f_i + h^2 f_i'' + \frac{h^4}{12} f_i^{(4)} + O(h^6).$$

Thus

$$\boxed{D_{xx} f_i = \frac{f_{i+1} - 2f_i + f_{i-1}}{h^2} = f_i'' + \frac{h^2}{12} f_i^{(4)} + O(h^4).}$$

This measures how the slope changes, or curvature. For $f = x^4$ at $x = 1$ with $h = 0.1$, it gives $(1.4641 - 2 + 0.6561)/0.01 = 12.02$, compared with the exact value 12. At $h = 0.05$ it gives 12.005: the error again decreases fourfold.

Small executable example (independent of the CFD run)

```
for h in (0.1, 0.05):
    f = lambda x: x**3
    forward = (f(1+h) - f(1)) / h
    backward = (f(1) - f(1-h)) / h
    centered = (f(1+h) - f(1-h)) / (2*h)
    curvature = ((1+h)**4 - 2 + (1-h)**4) / h**2
    print(h, forward, backward, centered, curvature)
```

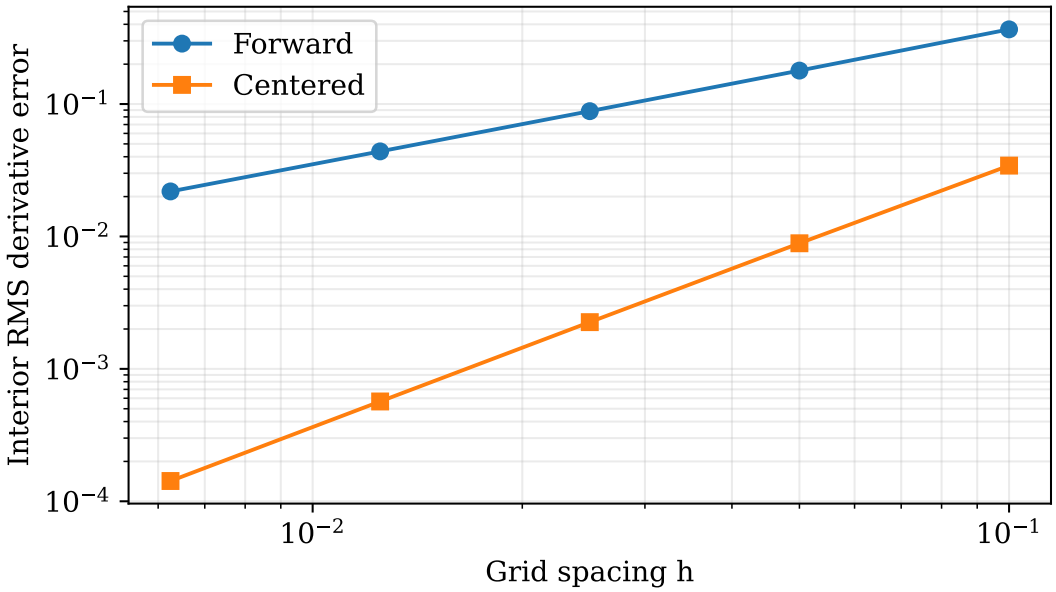
Check your understanding. For $f = x^2$, centered first and second derivatives are exact in exact arithmetic. Why is that example alone insufficient to demonstrate the observed order of a general derivative approximation?

Measuring accuracy in the notebook

For errors $e_i = D_h f_i - f'(x_i)$ at M interior nodes, the notebook uses the RMS norm $E_h = \sqrt{\sum_i e_i^2/M}$. If $E_h \approx Ch^p$, halving h gives

$$p_{\text{obs}} = \frac{\log(E_h/E_{h/2})}{\log 2}.$$

The cubic example above gives $\log(0.01/0.0025)/\log 2 = 2$. The notebook tests $\sin(\pi x)$ on five grids and compares its numerical derivative with $\pi\cos(\pi x)$; it then repeats a boundary check for e^x. Known derivatives reveal errors that a smooth-looking curve can hide.

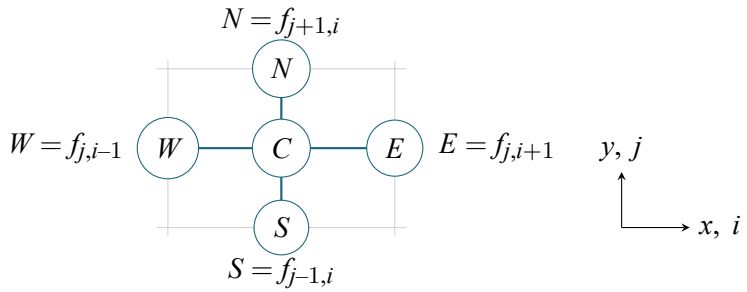


Executed derivative check. $f(x) = \sin(\pi x)$, $N = 11, 21, 41, 81, 161$, $h = 1/(N-1)$. Both methods are evaluated at interior nodes. The finest-pair observed orders are 1.005 (forward) and 1.995 (centered). This checks derivative accuracy; it does not establish cavity grid independence.

Predict, run, explain. Before running, predict the factors of two and four for forward and centered errors. After running, explain why the observed orders approach one and two. The finest grid is not automatically exact; eventually floating-point cancellation can compete with truncation error.

7.2 The two-dimensional stencil and array slices

A **stencil** is the set of nodes and weights used for one derivative approximation. The Laplacian $\nabla^2 f = f_{xx} + f_{yy}$ adds the curvatures along the two coordinate directions. We write $f_{j,i} = f(x_i, y_j)$ because NumPy indexes rows (y) before columns (x), although coordinates are written (x, y). At (x_i, y_j), use two neighbors in x and two in y. Equal spacing gives the five-point Laplacian. Diagonal nodes are not used by this stencil.



$$L_h f = \frac{E - 2C + W}{\Delta x^2} + \frac{N - 2C + S}{\Delta y^2} \xrightarrow{\Delta x = \Delta y = h} \frac{E + W + N + S - 4C}{h^2}.$$

Numerical check. For $f = x^2 + 3y^2$ at $(0.5, 0.5)$ and $h = 0.1$, $(C, E, W, N, S) = (1, 1.11, 0.91, 1.33, 0.73)$. Then $L_h f = (4.08 - 4)/0.01 = 8$, matching $f_{xx} + f_{yy} = 2 + 6$. This smooth polynomial is a stencil check, not a cavity solution.

One stencil, many nodes at once. In an $N_y \times N_x$ NumPy array:

Stencil part	Python slice	Shape
Center	`f[1:-1, 1:-1]`	$(N_y - 2, N_x - 2)$
East / west	`f[1:-1, 2:]` / `f[1:-1, :-2]`	Same
North / south	`f[2:, 1:-1]` / `f[:-2, 1:-1]`	Same

`1:-1` excludes the first and last entries; `2:` starts at index 2; `:-2` stops before the last two entries. NumPy applies arithmetic element by element to these equally shaped blocks. For $N = 65$, every interior block is 63×63.

Read `laplacian(phi, dx, dy)`. It allocates `zeros_like(phi)`, fills only the interior using the two second differences, and returns an array of the original shape. Its zero boundary entries are uncomputed placeholders, not a claim that the physical Laplacian is zero at a wall. The transport function writes the same interior stencil explicitly.

7.3 From the Poisson stencil to a coupled system

For equal spacing, $L_h\psi = -\omega$ becomes

$$4\psi_C - \psi_E - \psi_W - \psi_N - \psi_S = h^2\omega_C.$$

Solving this row for ψ_C gives a local update, but its four neighbors are also unknown. This is why discretization produces a *system* of equations rather than a collection of independent calculations.

For unequal spacings, the same rearrangement gives

$$\psi_{j,i}^{k+1} = \frac{(\psi_{j,i+1}^{k} + \psi_{j,i-1}^{k})\Delta y^2 + (\psi_{j+1,i}^{k} + \psi_{j-1,i}^{k})\Delta x^2 + \omega_{j,i}\Delta x^2\Delta y^2}{2(\Delta x^2 + \Delta y^2)}. \tag{25}$$

Here k counts inner sweeps; ω remains fixed throughout this Poisson solve.

A four-unknown problem you can solve by hand. Take two interior nodes in each direction in the unit square: $h = 1/3$. All boundary values of ψ are zero. Label the interior values a, b along the lower row and c, d along the upper row, and prescribe $\omega = -2$ at all four interior nodes. Then

$$\begin{bmatrix} 4 & -1 & -1 & 0 \\ -1 & 4 & 0 & -1 \\ -1 & 0 & 4 & -1 \\ 0 & -1 & -1 & 4 \end{bmatrix} \begin{bmatrix} a \\ b \\ c \\ d \end{bmatrix} = -\frac{2}{9} \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}.$$

The solution is $a = b = c = d = -1/9$. This example tests the Poisson equation only; its prescribed vorticity is not obtained from a complete no-slip cavity solve.

Jacobi iteration from zero. Update each node from the *previous sweep's* neighbors:

$$a^{k+1} = \tfrac{1}{4}(b^k + c^k - 2/9), \qquad b^{k+1} = \tfrac{1}{4}(a^k + d^k - 2/9),$$

with analogous rows for c, d. Symmetry makes all four entries identical in this example:

Sweep k	Each interior ψ	Error relative to $-1/9$
0	0	1/9
1	–1/18	1/18
2	–1/12	1/36
3	–7/72	1/72
∞	–1/9	0

Why the code uses `old = psi.copy()`. A complete snapshot makes the dependence on the old sweep explicit. In a nested point loop, immediately reusing updated neighbors would instead give a Gauss–Seidel-type sweep. In NumPy, the whole right-hand expression is evaluated before assignment, but `.copy()` also makes the algorithm unambiguous and preserves the earlier values.

This Poisson iteration index k is not time. Its inputs are a fixed current ω and fixed wall ψ. After this solve, the outer algorithm evolves ω and must solve a new Poisson problem.

Check your understanding. Starting from zero, compute the first Jacobi sweep in the four-node example, then identify the matching array slices. Does doubling the number of inner sweeps advance physical time? No: it only improves the Poisson solve at fixed vorticity.

8 Boundary conditions: closing the system

No slip means that the fluid velocity equals the solid-wall velocity; impermeability means no flow through the wall. The square cavity has $u = 1, v = 0$ on the moving top lid away from corners and $u = v = 0$ on the other walls. Set $\psi = 0$ on the entire boundary to impose impermeability. The tangential velocity conditions are encoded in boundary values of ω.

Derive the top-wall formula. Let P be the first interior point, a distance $h = \Delta y$ below the top wall w. Taylor expansion into the domain gives

$$\psi_P = \psi_w - h(\psi_y)_w + \frac{h^2}{2}(\psi_{yy})_w + O(h^3).$$

Along a straight wall where ψ is constant, $\psi_{xx} = 0$. Since $(\psi_y)_w = U_{\text{lid}}$ and $\omega_w = -\psi_{yy}|_w$,

$$\boxed{\omega_{\text{top}} \approx -\frac{2(\psi_P - \psi_w)}{\Delta y^2} - \frac{2U_{\text{lid}}}{\Delta y}.} \tag{26}$$

For this nondimensional cavity $U_{\text{lid}} = 1$ and $\psi_w = 0$. The same expansion at stationary walls gives

$$\omega_{\text{bottom}} \approx -2\psi_{1,i}/\Delta y^2, \tag{27}$$

$$\omega_{\text{left}} \approx -2\psi_{j,1}/\Delta x^2, \tag{28}$$

$$\omega_{\text{right}} \approx -2\psi_{j,N_x-2}/\Delta x^2. \tag{29}$$

The top uses $\psi_{N_y-2,i}$. These are the formulas in `apply_vorticity_boundary_conditions`. The local Taylor estimate for wall vorticity has an $O(h)$ neglected term; the second-order interior stencil alone does not prove second-order accuracy for the whole cavity solution.

Why wall vorticity changes during the solve. Even with a constant lid speed, ψ_P changes as the interior develops. Thus boundary vorticity must be recomputed when ψ changes. Setting all wall vorticity to zero would remove the no-slip physics.

Corners need an explicit convention. A uniform moving lid meets stationary sidewalls at a velocity discontinuity. The notebook averages adjacent wall-vorticity values at corners. Its velocity assignment leaves top-corner velocities at zero because the sidewall conditions are applied last. This is a discrete convention, not a smooth resolution of the continuum corner singularity. Interpret corner peaks and grid convergence accordingly.

Check your understanding. Initially $\psi = 0$ and $\Delta y = 1/64$. What top-wall vorticity is imposed away from corners? Answer: $\omega = -128$. Explain why this large boundary value is not the same as a large velocity.

9 Why the solver must iterate

Vorticity gets a time update. Forward Euler with central spatial differences gives

$$\omega_{j,i}^{n+1} = \omega_{j,i}^{n} + \Delta t \left[-u_{j,i}^{n} (D_x \omega^n)_{j,i} - v_{j,i}^{n} (D_y \omega^n)_{j,i} + \frac{1}{Re} (L_h \omega^n)_{j,i} \right]. \tag{30}$$

The interior spatial formulas are second order for smooth fields; forward Euler is first order in time. Boundaries and corner singularities can limit the observed global accuracy.

Vorticity is transported by u, v, but those velocities are recovered from ψ, and ψ is obtained from ω. The fields are coupled:

$$\omega \longrightarrow \underbrace{\nabla^2 \psi = -\omega}_{\text{spatial solve}} \longrightarrow (u, v) \longrightarrow \underbrace{\omega_t = -u\omega_x - v\omega_y + Re^{-1} \nabla^2 \omega}_{\text{time update}}.$$

No one of these fields can be treated as a permanently known input while solving for the others.

Two different repetition processes

Process	What stays fixed?	Why repeat?
Inner Jacobi sweeps (k)	Current vorticity and wall ψ	Each unknown streamfunction value depends on unknown neighbors.
Outer time steps (n)	Geometry, Re, and lid motion	Vorticity, velocity, and wall vorticity evolve together toward the target state.

A direct linear solver could replace the inner Jacobi loop. Thus iteration is not a mathematical requirement for every Poisson solver; it is the notebook's chosen solution method. The outer loop here integrates an initial transient toward steady flow. For a genuinely unsteady target, one would retain the time history instead of requiring a steady state.

Algorithm matching `run_cavity`

1. Initialize interior $\psi = \omega = 0$; apply wall-vorticity conditions.
2. With current ω, take 60 Jacobi sweeps for ψ, keeping boundary $\psi = 0$.
3. Recompute wall vorticity using this ψ and the prescribed lid speed.
4. Calculate interior $u = D_y \psi$, $v = -D_x \psi$; impose wall velocities.
5. Advance interior ω by one forward-Euler step using these velocities.
6. Reapply wall vorticity and, at reporting steps, record relative vorticity change.
7. Repeat steps 2–6 for the specified number of time steps. Finally use 120 Poisson sweeps and recover wall vorticity and velocity again.

Teaching settings. The default example uses 65×65 nodes, $Re = 100$, $\Delta t = 0.001$, and 10,000 steps, corresponding to dimensionless $t = 10$. The notebook reports progress but runs to a fixed step count; it does not terminate automatically when a tolerance is reached. Sixty sweeps are a computational setting, not proof of Poisson convergence.

Check your understanding. If you increase the number of Poisson sweeps without taking any additional outer time steps, what error can decrease and what physical evolution has not occurred?

10 Convergence and stability are different checks

Iteration change. The notebook reports

$$R_{\Delta\omega} = \frac{\|\omega^{n+1} - \omega^n\|_2}{\|\omega^{n+1}\|_2 + 10^{-30}}. \tag{31}$$

A small update is useful evidence, but a smaller Δt can make it small even when the steady PDE defect remains large. In forward Euler, the interior update divided by Δt is the current transport right-hand side.

Equation defects. For stronger checks, evaluate the interior fields with consistent stencils:

$$r_\psi = L_h\psi + \omega, \tag{32}$$

$$r_\omega = uD_x\omega + vD_y\omega - Re^{-1}L_h\omega \quad \text{(steady target)}, \tag{33}$$

$$r_{\text{div}} = D_x u + D_y v. \tag{34}$$

Report a norm such as RMS or maximum absolute value, the grid and normalization, and the wall-velocity error. These are recommended diagnostics; the introductory solver's existing progress printout only reports $R_{\Delta\omega}$.

Explicit time-step restrictions. For the diffusion-only forward-Euler update on this grid, the standard restriction is

$$\frac{\Delta t}{Re}\left(\frac{1}{\Delta x^2} + \frac{1}{\Delta y^2}\right) \le \frac{1}{2}. \tag{35}$$

For $h = 1/64$ and $Re = 100$, this gives $\Delta t \le Re\, h^2/4 \approx 0.00610$. The notebook's 0.001 passes this *diffusion-only* check. Also monitor advective Courant numbers, for example

$$C_x = \max|u|\Delta t/\Delta x, \qquad C_y = \max|v|\Delta t/\Delta y.$$

These checks do not by themselves prove stability of the coupled solver. Forward Euler with centered differences is unstable for pure advection; diffusion, resolution, boundary treatment, and the complete discretization matter. At higher Reynolds numbers, a different time integrator or convection scheme may be needed.

Three practical comparisons. First tighten the Poisson solve and increase integration time at a fixed grid. Next reduce Δt to check temporal sensitivity. Then refine the grid and compare quantities of interest. Do not confuse a converged algebraic solve with an accurate continuum solution. A small ML loss has the same limitation: it certifies only what the chosen loss actually measures.

Check your understanding. A run's reported update falls when Δt is divided by ten. What additional evidence is needed before calling the flow steady?

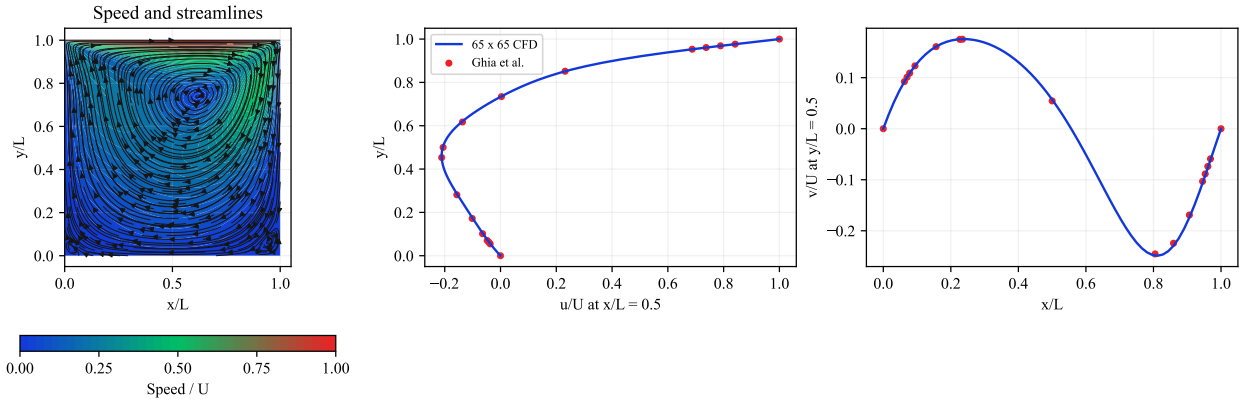
11 Benchmarks and the Ghia comparison

Couette flow gives a linear velocity profile for a simple shear-driven problem. Plane Poiseuille flow gives a parabolic profile for a fully developed pressure-driven channel. Both help isolate basic derivative and boundary-condition errors. The lid-driven cavity combines recirculation, walls, and nonlinear convection, making it a more demanding benchmark.

Compare quantities, not only pictures. Ghia, Ghia, and Shin [1] tabulate u/U on the vertical centerline $x/L = 0.5$ and v/U on the horizontal centerline $y/L = 0.5$. The notebook interpolates the computed profiles to those sampling positions and calculates

$$E_u = \frac{\|u_{\text{interp}} - u_{\text{Ghia}}\|_2}{\|u_{\text{Ghia}}\|_2}, \qquad E_v = \frac{\|v_{\text{interp}} - v_{\text{Ghia}}\|_2}{\|v_{\text{Ghia}}\|_2}. \tag{36}$$

These are errors at benchmark sample points, not full-domain errors. Use the same Reynolds number, velocity normalization, centerline locations, and sign convention.



Fresh teaching computation. Existing notebook solver, $Re = 100$, 65×65 nodes, $\Delta t = 0.001$, 10,000 steps, 60 inner sweeps per step. Markers are the notebook's Ghia table values. The relative centerline errors are $E_u = 0.343\%$ and $E_v = 1.771\%$; the last reported relative vorticity change is 6.02×10^{-7}. The associated `validation.json` records these diagnostics. This single-grid comparison is not grid-independent certification.

Interpret the flow. The rightward lid drives a dominant clockwise circulation. Near the vertical centerline the lower part of the cavity has negative u, showing the return flow. Weak corner eddies may be difficult to resolve or display: streamline density, grid resolution, and convergence all affect their appearance. A plausible vortex alone is not validation.

From fields to ML tensors. A field of shape (N_y, N_x) can become one channel of an array with channels (u, v, ω), and multiple cases add a batch dimension. Preserve axes, scales, boundary conventions, and parameter labels. Before using these arrays as training targets, qualify their numerical accuracy; a network trained on inaccurate CFD can reproduce its errors.

Check your understanding. Why can a visually smooth predicted vortex still have a poor centerline error or violate the wall condition?

12 Optional pressure recovery: full derivation

Use dimensional variables on this page, omitting the subscript d until the final scaling step. Assume constant ρ, ν, smooth fields, no body force, and $u_x + v_y = 0$ for all time:

$$u_t + uu_x + vu_y = -p_x/\rho + \nu(u_{xx} + u_{yy}),$$
$$v_t + uv_x + vv_y = -p_y/\rho + \nu(v_{xx} + v_{yy}).$$

1. Differentiate and add. Apply ∂_x to horizontal momentum and ∂_y to vertical momentum:

$$\partial_t(u_x + v_y) + \partial_x(uu_x + vu_y) + \partial_y(uv_x + vv_y) = -\rho^{-1}\nabla^2 p + \nu\nabla^2(u_x + v_y).$$

The time and viscous terms vanish because the divergence is identically zero. This is not a steady-flow or zero-viscosity assumption: the derivation applies to unsteady viscous incompressible flow too.

Keep all four viscous derivatives before canceling. Write $u_x = \partial u/\partial x$ and $v_y = \partial v/\partial y$. For smooth interior fields, mixed partial derivatives commute:

$$\begin{aligned}\partial_x\nabla^2 u + \partial_y\nabla^2 v &= \partial_x u_{xx} + \partial_x u_{yy} + \partial_y v_{xx} + \partial_y v_{yy} \\ &= \partial_{xx}u_x + \partial_{yy}u_x + \partial_{xx}v_y + \partial_{yy}v_y \\ &= \partial_{xx}(u_x + v_y) + \partial_{yy}(u_x + v_y) \\ &= \nabla^2 \underbrace{(u_x + v_y)}_{0} = 0.\end{aligned}$$

For example, $\partial_x u_{yy} = \partial_{yy}u_x$ and $\partial_y v_{xx} = \partial_{xx}v_y$ allow the middle two terms to be grouped with the correct partners. One may not replace $\partial_{xx}u_x + \partial_{yy}v_y$ alone by $\nabla^2(u_x + v_y)$: two cross terms would be missing.

A quick counterexample to canceling individual pairs. Let $u = x^3 y$, $v = -\frac{3}{2}x^2y^2$. Then $u_x + v_y = 0$, but

$$\partial_{xx}u_x + \partial_{yy}v_y = 6y, \qquad \partial_{yy}u_x + \partial_{xx}v_y = -6y.$$

Only the complete four-term sum is zero. The time term vanishes separately because $\partial_t(u_x + v_y) = 0$. These smooth interior identities do not imply that every discrete derivative or moving-lid corner value satisfies them exactly. Taking the *curl* instead gives $\nu\nabla^2\omega$, which remains in vorticity transport.

2. Expand the convective terms.

$$\partial_x(uu_x + vu_y) = u_x^2 + uu_{xx} + v_x u_y + vu_{yx},$$
$$\partial_y(uv_x + vv_y) = u_y v_x + uv_{xy} + v_y^2 + vv_{yy}.$$

Their sum is

$$\begin{aligned} &u_x^2 + 2u_y v_x + v_y^2 + u(u_{xx} + v_{xy}) + v(u_{yx} + v_{yy}) \\ &= u_x^2 + 2u_y v_x + v_y^2 + u\partial_x(u_x + v_y) + v\partial_y(u_x + v_y) \\ &= u_x^2 + 2u_y v_x + v_y^2. \end{aligned}$$

3. Solve for the pressure Laplacian.

$$\boxed{\nabla^2 p = -\rho\left[u_x^2 + 2u_y v_x + v_y^2\right]} \quad \text{(dimensional)}.$$

Since $v_y = -u_x$, an equivalent expression is

$$\nabla^2 p = 2\rho(u_x v_y - u_y v_x).$$

4. Scale the equation. With $p_d = p_{\text{ref}} + \rho U^2 p$, $(x_d, y_d) = L(x, y)$, and $(u_d, v_d) = U(u, v)$, both sides carry the factor $\rho U^2/L^2$. Canceling it gives

$$\boxed{\nabla^2 p = -\left[u_x^2 + 2u_y v_x + v_y^2\right]} \quad \text{(dimensionless)}.$$

There is no explicit *Re* in this interior source, but the velocity gradients depend on the viscous solution. Pressure boundary data also depend on viscosity. The equation alone is not a complete pressure boundary-value problem.

12.1 Pressure walls, compatibility, and the arbitrary constant

Return to dimensionless variables. Rearrange momentum and project it onto the outward unit normal $\boldsymbol{n}$:

$$\boxed{\partial_n p = \boldsymbol{n} \cdot \left[-\boldsymbol{u}_t - (\boldsymbol{u} \cdot \nabla)\boldsymbol{u} + Re^{-1}\nabla^2 \boldsymbol{u} \right] = g.}$$

The wall pressure gradient therefore follows from momentum. Imposing zero normal pressure gradient on every wall is not generally justified.

Smooth straight cavity walls. Away from the lid corners, a wall with no penetration and constant tangential speed has zero normal velocity for all time. The normal time derivative and the normal convective acceleration vanish at that wall. Thus

$$p_y = Re^{-1}\nabla^2 v \text{ on horizontal walls}, \qquad p_x = Re^{-1}\nabla^2 u \text{ on vertical walls}.$$

From continuity and $\omega = v_x - u_y$,

$$\omega_x = v_{xx} - u_{yx} = v_{xx} + v_{yy} = \nabla^2 v, \qquad -\omega_y = -v_{xy} + u_{yy} = u_{xx} + u_{yy} = \nabla^2 u.$$

The outward-normal data are therefore

Wall	$\boldsymbol{n}$	$g = \partial_n p$
Top	$(0, 1)$	$+Re^{-1}\omega_x$
Bottom	$(0, -1)$	$-Re^{-1}\omega_x$
Right	$(1, 0)$	$-Re^{-1}\omega_y$
Left	$(-1, 0)$	$+Re^{-1}\omega_y$

Use consistent boundary-aware derivatives. At the discontinuous lid corners, use the chosen discrete corner treatment; smooth-wall Taylor arguments do not apply exactly there.

Compatibility. For $\nabla^2 p = S$ and $\partial_n p = g$, the divergence theorem requires

$$\boxed{\int_\Omega S \, \mathrm{d}A = \int_{\partial\Omega} g \, \mathrm{d}s.}$$

An exact velocity solution satisfies this relation. Discretization errors can cause a mismatch; measure and report it. Any numerical source correction must be documented, because it changes the discrete recovery problem.

Gauge. Adding a constant to p leaves all gradients unchanged. Choose a zero mean or one reference pressure. After a compatible solve, $p \leftarrow p - \bar{p}$ fixes the gauge. It cannot repair incompatible source and boundary data. On nonuniform grids, use the intended volume weights to define the mean.

Body-force extension. If momentum includes a force per unit mass $\boldsymbol{f}$, the dimensionless source gains $+\nabla \cdot \boldsymbol{f}$ and the wall data gain $+\boldsymbol{n} \cdot \boldsymbol{f}$. Constant gravity, for example, can affect boundary data even though its divergence is zero.

12.2 Pressure recovery in practice and method references

Complete sequence. Obtain the velocity field; calculate consistent gradients; form $S = -[u_x^2 + 2u_y v_x + v_y^2]$ and the momentum-based wall data; check compatibility; solve the Neumann problem with one gauge; check both pressure-Poisson and original momentum residuals. Unsteady momentum checks need the velocity time derivative from the time history.

The interior Jacobi update for pressure on a rectangular grid would be

$$p_C^{k+1} = \frac{(p_E^k + p_W^k)\Delta y^2 + (p_N^k + p_S^k)\Delta x^2 - S_C \Delta x^2 \Delta y^2}{2(\Delta x^2 + \Delta y^2)}.$$

The source sign is negative. The streamfunction update has a positive omega term because its Poisson source is $-\omega$. Boundary rows or ghost values and a gauge are still necessary; this interior formula alone is not a pressure solver.

For an assembled all-Neumann system $Ap = b, A\mathbf{1} = 0$. If $z^T A = 0$, compatibility requires $z^T b = 0$. The left-nullspace weights depend on the discretization and need not be uniform. A small pressure-Poisson residual also does not guarantee small residuals in both momentum components if the input velocity is inaccurate.

Analytic checks. Couette flow $u = y, v = 0$ gives $S = 0$ and constant pressure in the force-free model. Solid-body rotation $u = -\Omega y, v = \Omega x$ gives $S = 2\Omega^2$ and $p = \Omega^2(x^2 + y^2)/2 + C$. The latter checks signs and factors; it is not a no-slip cavity solution.

Attribution for this notebook. The existing code uses centered finite differences, a Jacobi streamfunction solve, explicit Euler vorticity stepping, and wall vorticity obtained from a local Taylor expansion. It compares with Ghia's data; it does not implement CSI-MG. The derivations here are independently worked from the stated equations, not copied from either article. This section explains pressure recovery; it does not add a pressure solver to the notebook.

Summary. The six outcomes listed on page 1 are the checklist for this week; test each against the checkpoints above before moving to the notebooks.

Optional derivation exercise. Reproduce the curl-of-momentum calculation on page 7, identifying the exact use of incompressibility. Then derive the moving-wall vorticity formula from a Taylor expansion.

References and authorship scope

[1] U. Ghia, K. N. Ghia, and C. T. Shin (1982). High-Re solutions for incompressible flow using the Navier–Stokes equations and a multigrid method. *Journal of Computational Physics*, 48, 387–411. Benchmark data used by the existing notebook.

[2] F. M. White and H. Xue (2022). *Fluid Mechanics*, 9th ed. McGraw Hill. Background on conservation laws and incompressible flow.

[3] J. D. Anderson (1995). *Computational Fluid Dynamics: The Basics with Applications*. McGraw Hill. Background on discretization and numerical solution.

[4] MIT OpenCourseWare, *Notes on Numerical Methods in Ocean Modeling*, Lecture 1, pp. 4–12. Introductory Taylor expansions and finite-difference equations. Course notes (PDF).

[5] NASA Glenn Research Center, *Examining Spatial (Grid) Convergence*. Guidance on refinement and observed order. NASA grid-convergence guide.

[6] A. Thom (1933). *The flow past circular cylinders at low speeds*. Proceedings of the Royal Society of London, Series A, 141, 651–669. Historical numerical-flow reference. doi:10.1098/rspa.1933.0146.

Explanations and derivations were independently written for FlowMLLab's introductory cavity example, using the requested topic checklist. The figure is a fresh execution of FlowMLLab's own solver. No restricted course text, solution, figure, or code was incorporated. No ML split or calibration is involved in this numerical-foundations extension; its baseline is the stated Ghia centerline comparison.

B1 Reading the code: grid and Poisson solve

The following excerpts are from the introductory solver in `03_cavity_ghia.ipynb`; comments and line wrapping are added for teaching. Execute the definitions before calling `run_cavity`. The separate pressure-validation cells at the start of that notebook are not required to understand these functions.

`build_grid(N)`: positions before fields

```
x = np.linspace(0.0, 1.0, N)
y = np.linspace(0.0, 1.0, N)
X, Y = np.meshgrid(x, y)
dx = x[1] - x[0]
dy = y[1] - y[0]
```

`linspace` includes both endpoints, so N nodes give $N-1$ intervals and $\Delta x = 1/(N-1)$. With the default `meshgrid` indexing, `X[j,i]=x[i]` and `Y[j,i]=y[j]`. `X,Y` are coordinate arrays for plotting; they are not the velocity fields. The function returns all six arrays/scalars. For $N = 65$, `x.shape=(65,)` and `X.shape=(65,65)`.

`solve_streamfunction_poisson`: implement the algebra

```
dx2 = dx**2
dy2 = dy**2
denom = 2.0 * (dx2 + dy2)
for _ in range(iterations):
    old = psi.copy()
    psi[1:-1, 1:-1] = (
        (old[1:-1, 2:] + old[1:-1, :-2]) * dy2
        + (old[2:, 1:-1] + old[:-2, 1:-1]) * dx2
        + omega[1:-1, 1:-1] * dx2 * dy2
    ) / denom
    psi[0, :] = 0.0
    psi[-1, :] = 0.0
    psi[:, 0] = 0.0
    psi[:, -1] = 0.0
```

The first neighbor sum is east plus west; the second is north plus south. Multiplying the finite-difference equation by $\Delta x^2 \Delta y^2$ explains the apparently crossed weights: east/west gets `dy2` and north/south gets `dx2`. The *positive* omega term in the numerator is correct for $\nabla^2 \psi = -\omega$.

`range(iterations)` performs a fixed number of sweeps; the underscore means that the loop counter is unused. The four assignments restore the Dirichlet wall values after each sweep. There is no residual-based stopping test inside this function. The returned `psi` is then used to calculate velocity.

B2 Reading the code: wall vorticity and velocity

`apply_vorticity_boundary_conditions`

```
omega[0, 1:-1] = -2.0 * psi[1, 1:-1] / dy**2
omega[-1, 1:-1] = (
    -2.0 * psi[-2, 1:-1] / dy**2 - 2.0 * U / dy
)
omega[1:-1, 0] = -2.0 * psi[1:-1, 1] / dx**2
omega[1:-1, -1] = -2.0 * psi[1:-1, -2] / dx**2
```

Line 1 updates the bottom, lines 2–4 the top, then the left and right walls. Index `-1` means the last row or column and `-2` the next-to-last one. Since `y` increases from 0 to 1, the last row is the top wall. The top-wall expression contains the additional moving-lid term derived on page 13.

Slices exclude corners because two different walls meet there. The notebook sets each corner to the average of its adjacent wall values, for example

```
omega[-1, 0] = 0.5 * (omega[-1, 1] + omega[-2, 0])
```

This function updates the supplied array in place and returns it. The caller's `omega` therefore changes; it is not an independent new array.

`compute_velocity`: derivatives of the new streamfunction

```
u = np.zeros_like(psi)
v = np.zeros_like(psi)
u[1:-1, 1:-1] = (
    psi[2:, 1:-1] - psi[:-2, 1:-1]
) / (2.0 * dy)
v[1:-1, 1:-1] = -(
    psi[1:-1, 2:] - psi[1:-1, :-2]
) / (2.0 * dx)
u[0, :] = 0.0
u[-1, :] = U
u[:, 0] = 0.0
u[:, -1] = 0.0
```

Horizontal velocity uses a difference in the *first* index because $u = \psi_y$. Vertical velocity uses a difference in the second index and a minus sign because $v = -\psi_x$. The notebook also explicitly sets all four boundaries of `v` to zero.

The final two assignments for `u` overwrite the top corners with zero. Changing assignment order can change the corner convention. For the lecture's nondimensional $Re = 100$ case, keep `U=1`; changing only `U` while holding the diffusion coefficient $1/Re$ fixed changes the effective Reynolds number based on the actual lid speed.

B3 Reading the code: one vorticity time step

`advance_vorticity` takes the current fields and returns both updated vorticity and the old snapshot. The snapshot is needed for explicit stepping and the change diagnostic.

```
old = omega.copy()
dwdx = (old[1:-1, 2:] - old[1:-1, :-2]) / (2.0 * dx)
dwdy = (old[2:, 1:-1] - old[:-2, 1:-1]) / (2.0 * dy)
lap_omega = (
    (old[1:-1, 2:] - 2.0*old[1:-1, 1:-1]
     + old[1:-1, :-2]) / dx**2
    + (old[2:, 1:-1] - 2.0*old[1:-1, 1:-1]
       + old[:-2, 1:-1]) / dy**2
)
convection = u[1:-1, 1:-1]*dwdx + v[1:-1, 1:-1]*dwdy
diffusion = (1.0 / Re) * lap_omega
omega[1:-1, 1:-1] = (
    old[1:-1, 1:-1] + dt * (-convection + diffusion)
)
return omega, old
```

Code quantity	Mathematical meaning
`dwdx, dwdy`	$D_x\omega^n, D_y\omega^n$
`lap_omega`	$L_h\omega^n$
`convection`	$u^n D_x\omega^n + v^n D_y\omega^n$
`diffusion`	$Re^{-1} L_h\omega^n$
`-convection + diffusion`	Approximation to $\partial\omega/\partial t$

The transport equation has convection on the left. Moving it to the right explains the minus sign in the update. Multiplying the right-hand side by `dt` converts a rate of change into an increment. There is no division by `dt` in the final explicit update.

One-node calculation. Suppose $\omega^n = -2$, $u = 0.4$, $v = -0.1$, $D_x\omega = 3$, $D_y\omega = -2$, and $L_h\omega = 20$. At $Re = 100$ and $\Delta t = 0.001$, convection is 1.4, diffusion is 0.2, and

$$\omega^{n+1} = -2 + 0.001(-1.4 + 0.2) = -2.0012.$$

These supplied local numbers illustrate the arithmetic, not a measured cavity point. For a real node, the surrounding old-field values determine all three derivatives.

Only interior entries are advanced. Boundary entries are imposed separately by the wall-vorticity function, after the update. Mixing new and old neighbor values inside this explicit stencil would implement a different numerical method.

B4 Reading the code: orchestration and diagnostics

`run_cavity` creates the grid, initializes zero arrays, and imposes wall vorticity before entering this loop. This excerpt follows the actual call order; the long diagnostic expression is wrapped for readability.

```
for step in range(1, steps + 1):
    psi = solve_streamfunction_poisson(
        psi, omega, dx, dy, iterations=poisson_iters)
    omega = apply_vorticity_boundary_conditions(
        psi, omega, U, dx, dy)
    u, v = compute_velocity(psi, U, dx, dy)
    omega, old_omega = advance_vorticity(
        omega, u, v, Re, dt, dx, dy)
    omega = apply_vorticity_boundary_conditions(
        psi, omega, U, dx, dy)
    if step % report_every == 0:
        residual = (np.linalg.norm(omega - old_omega)
                    / (np.linalg.norm(omega) + 1e-30))
        residual_steps.append(step)
        residual_values.append(residual)
```

`range(1, steps+1)` includes the final requested step. The modulo condition reports every `report_every` steps; it does not skip evolution between reports. For a two-dimensional array, this use of `np.linalg.norm` gives the Frobenius norm, the square root of the sum of squared entries.

After the loop the code performs another Poisson solve with `2*poisson_iters`, reapplies wall vorticity, and recomputes velocity. It returns a dictionary containing coordinates, fields, diagnostics, and run parameters. Access a field as `result["u"]`. The final adjustment means the last printed change diagnostic is not a fresh simultaneous residual evaluation of every returned equation.

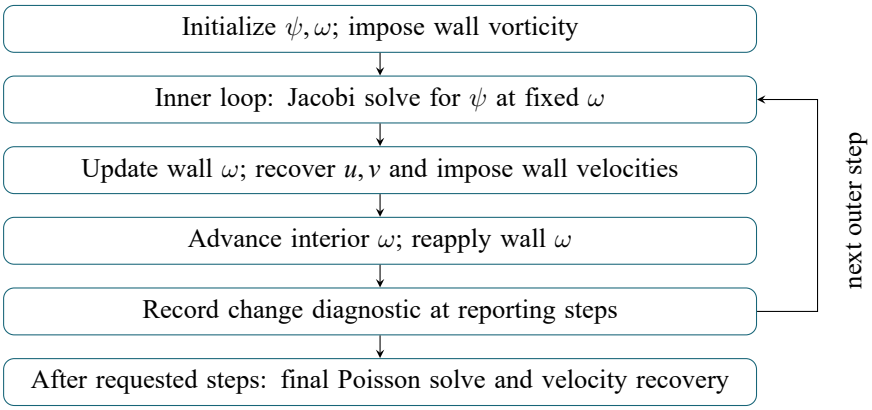
`centerline_profiles` and `ghia_errors`

```
mid = len(x) // 2
u_center = u[:, mid] / U
v_center = v[mid, :] / U
u_interp = np.interp(GHIA_Y, y, u_center)
v_interp = np.interp(GHIA_X, x, v_center)
err_u = np.linalg.norm(u_interp-GHIA_U) / np.linalg.norm(GHIA_U)
err_v = np.linalg.norm(v_interp-GHIA_V) / np.linalg.norm(GHIA_V)
```

For the odd uniform grid $N = 65$, index 32 lies at 0.5. `u[:,mid]` fixes x and varies y; `v[mid,:]` fixes y and varies x. `np.interp` samples these computed curves at the benchmark coordinates. An even grid has no node exactly at 0.5, so this simple index selection would need interpolation in the transverse direction too.

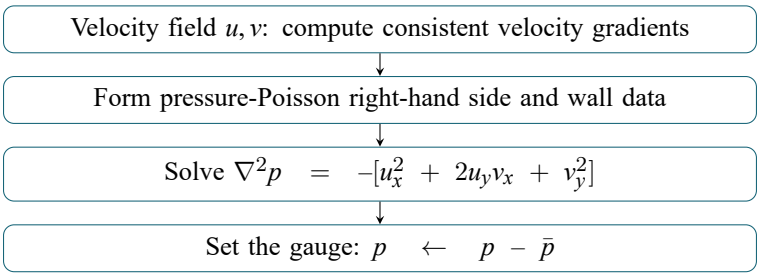
C Visual guide: the solver and pressure extension

The actual loop has two levels. The diagram retains the original lecture's algorithm overview while making the fixed-step behavior and boundary-update order explicit.



There is no tolerance-triggered exit in the current teaching loop. Inspect equation defects and benchmark agreement as well as changes between iterates. A declining update alone does not demonstrate grid independence.

Optional pressure recovery. This separate path begins with a computed velocity field; it is not a feedback step required by the streamfunction–vorticity solver.



The gauge removes an arbitrary constant. Correct wall data and Neumann compatibility still matter, as explained on page 17. The introductory `run_cavity` return dictionary does not contain a pressure field; the notebook's separate pressure-validation material is an extension with its own run settings.

Suggested classroom sequence. Derive one stencil, do one update by hand, identify the matching array slice, and then run the full example. Ask students to predict the effect of a sign error before showing a plot.

D Coverage of the original lecture

This expanded edition reorganizes the original lecture; it is not a verbatim copy. The table maps all sixteen original sections. Earlier shortened definitions and visual explanations have been restored or expanded. The original PDF remains available unchanged at its baseline Git commit, linked in the accompanying coverage note.

Original section	Where to find it here
1. Purpose of this lecture	Opening page: learning question and three notebooks
2. What is a fluid?	Section 1: definition, examples, and variable glossary
3. Continuum viewpoint	Section 1: molecular scales, fields, and arrays
4. Eulerian description	Section 1: fixed positions versus particle following
5. Velocity and material derivative	Section 1: chain rule and local/convective acceleration
6. Conservation of mass	Section 2: general and incompressible continuity
7. Momentum: Navier–Stokes	Section 2: vector and both component equations
8. Reynolds number	Section 3: scales, physical interpretation, $Re = 100$
9. Boundary conditions	Section 8 and B2: no slip, impermeability, corner convention
10. Streamfunction–vorticity	Sections 4–6: definitions, proof, transport derivation
11. Finite differences	Sections 7–7.3: Taylor series, examples, stencil, algebraic system
12. Residuals and convergence	Section 10 and B4: diagnostics, limits, ML connection
13. Couette, Poiseuille, cavity	Section 11; Couette example also in Section 4
14. Ghia validation	Section 11 and B4: profiles, interpolation, errors
15. Optional pressure recovery	Section 12 and C: equation, boundary data, gauge
16. End-of-week outcomes	Section 12 and the checkpoints throughout

Figures. The old streamline illustration is replaced by a fresh computation with centerline comparisons (page 16). The five-point stencil has a new labeled drawing in Section 7.2. The solver and pressure-recovery diagrams are redrawn in C. Their instructional roles are preserved, while the solver diagram now matches the code's fixed-step implementation.

Scientific clarifications. The original Eq. (15) is proved in Section 5; equation numbers have otherwise changed. Dimensionless viscosity is distinguished from physical viscosity. Pressure recovery is stated with its boundary conditions and gauge. The claim that iteration stops at a residual tolerance is replaced by the actual fixed-step behavior. These corrections retain the topics while improving accuracy. **Code scope.** The notebook's numerical algorithm is unchanged. B1–B4 explain its existing operations; the arithmetic examples in Section 7 and the code walkthrough are explicitly identified as teaching checks.